Universidade do Minho
Licenciatura em Engenharia Informática

RELATÓRIO

Laboratórios de Informática III

Fase 1

Índice:

1. Introdução ---------- 2
2. Sistema ---------- 3
3. Discussão ---------- 6
3.1. Metodologia de Medição ---------- 9
3.2. Análise da discrepância de resultados---------- 9
4. Conclusão ---------- 10

Novembro, 2025

Trabalho realizado por:

Ana Patrícia da Silva Machado (*a111661*)
Alexandra Cerqueira da Luz das Neves (*a110903*)
Carolina Francisca Vieira Dias (*a111179*)

1. INTRODUÇÃO

O presente relatório detalha o desenvolvimento do trabalho prático da Unidade Curricular de Laboratórios de Informática III, da Universidade do Minho. Este projeto tem como principal objetivo a implementação de um programa robusto, em linguagem C, capaz de processar e analisar diferentes ficheiros de dados no formato CSV, que simula a gestão de um sistema de informação relacionado com aeroportos, aeronaves, voos, passageiros e reservas.

O programa foi desenvolvido de forma modular, visto que está dividido em diferentes ficheiros *.c* e *.h*, cada um responsável por uma parte específica do projeto. Esta estrutura permite garantir uma melhor organização, legibilidade e manutenção do código, seguindo os princípios de **modularização** e **encapsulamento**.

Cada módulo trata um conjunto específico de funcionalidades, como a leitura dos ficheiros de dados, o armazenamento da informação em *HashTables* e a execução das várias queries pedidas. Para isto, foi utilizada a biblioteca **GLib**, que fornece estruturas de dados dinâmicas (neste caso, dos aeroportos, aeronaves e voos), como **GHashTables**, fundamentais para armazenar e aceder rapidamente à informação pretendida. Adicionalmente, o programa utiliza também outras bibliotecas padrão da linguagem C, como *stdio.h*, *stdlib.h*, *string.h*, *ctype.h*, *time.h* e *stdbool.h*.

O funcionamento geral do programa baseia-se na leitura de um conjunto de ficheiros (*airports.csv*, *aircrafts.csv* e *flights.csv*) e na execução das queries, gerando assim os respetivos resultados. Cada query produz um ficheiro de saída com o resultado correspondente, o que garante que o programa continue a sua execução, mesmo em caso de erro ou de queries não implementadas.

Durante o desenvolvimento deste trabalho foram tidas em conta boas práticas de gestão de memória e verificação de erros, recorrendo assim à ferramenta **Valgrind** para detetar possíveis fugas de memória.

Foram utilizados o **Git** e o **GitHub** que permitem um trabalho colaborativo e o acompanhamento das alterações de cada membro do grupo ao longo do projeto.

Este relatório descreve em detalhe a estrutura do código, as principais funcionalidades implementadas, as decisões de implementação tomadas e os testes realizados para validar o correto funcionamento do programa.

2. SISTEMA

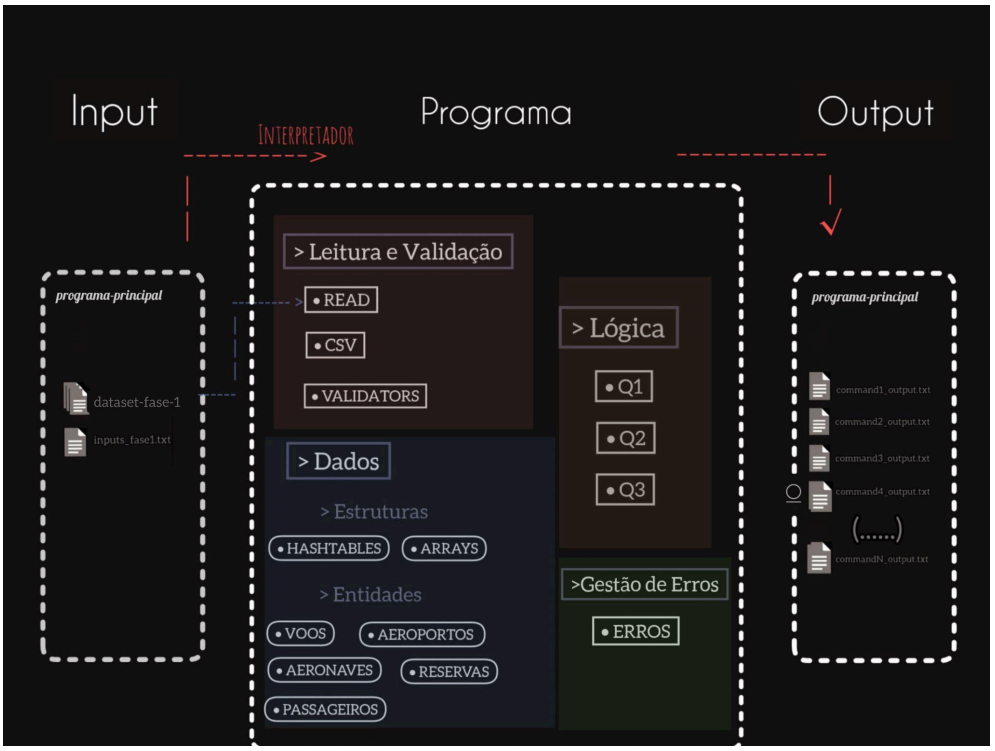

Fig 1 - Diagrama da Arquitetura do Sistema

O programa foi desenvolvido em linguagem C, seguindo uma estrutura modular, de forma a garantir uma clara separação entre os diferentes componentes do sistema.

O **funcionamento do sistema** baseia-se na leitura de cinco ficheiros CSV (*airports.csv*, *aircrafts.csv, flights.csv, passengers.c* e *reservations.c*), que contêm a informação necessária sobre os aeroportos, aeronaves, voos, passageiros e reservas, respetivamente. Este conteúdo será processado e armazenado em **estruturas de dados dinâmicas**, providenciadas pela biblioteca **GLib**, nomeadamente através da utilização de **GHashTable**, que permite uma pesquisa e acesso eficientes aos registos.

A **organização do projeto** contém um diretório chamado trabalho-prático que contém pastas e ficheiros, cada um com uma funcionalidade específica:

- **include/**: contém os ficheiros de cabeçalho (*.h*), que definem as estruturas de dados, protótipos das funções e constantes globais;
- **src/:** contém os ficheiros *.c*, responsáveis pela implementação das funcionalidades de leitura dos ficheiros, validação dos dados e execução das queries;
- **resultados/**: armazena os ficheiros de saída gerados após a execução das queries;
- **Makefile**, **programa-principal** e **programa-testes**: gerados automaticamente após a compilação.

A compilação e execução são automatizadas através do **Makefile**, garantindo compatibilidade com o ambiente de testes usado nesta Unidade Curricular. Este ficheiro centraliza todas as regras necessárias para compilar os diferentes módulos do projeto, gerar os executáveis correspondentes e organizar os ficheiros objeto (*.o*). Além disso, permite a execução de comandos auxiliares, como a execução do programa principal com os datasets fornecidos e a comparação dos resultados obtidos com os esperados, assegurando assim uma integração eficiente e consistente do sistema.

O **ficheiro *programa-principal.c*** tem como função coordenar a execução do programa. Este módulo:

- lê os ficheiros de dados e cria as HashTables correspondentes;
- lê um ficheiro de comandos que contém as queries a executar;
- chama as funções associadas a cada query;
- gera ficheiros de saída com os resultados em formato .txt, guardados na pasta *resultados/*.

O **ficheiro *programa-testes.c*** foi criado para auxiliar a validação das funções desenvolvidas, permitindo verificar o comportamento do sistema e detetar eventuais erros durante o desenvolvimento. Este módulo:

- executa um conjunto de **testes unitários e funcionais** sobre as principais funções desenvolvidas;
- verifica a consistência das estruturas de dados criadas (como as **HashTables** da GLib);
- compara os resultados obtidos com os valores esperados, permitindo identificar erros ou comportamentos inesperados;
- gera relatórios simples em formato *.txt* com o estado dos testes, que podem ser consultados para confirmar a fiabilidade do sistema.

Cada query foi implementada num módulo separado (*q1.c*, *q2.c* e *q3.c*) e possui o seu ficheiro de cabeçalho próprio (*q1.h*, *q2.h* e *q3.h*). Esta abordagem assegura o **encapsulamento** e a **reutilização de código**, permitindo que futuras extensões (possibilidade de novas queries na fase 2 do projeto) sejam facilmente integradas sem afetar o trabalho anteriormente elaborado.

Para além da biblioteca GLib, o sistema utiliza várias bibliotecas padrão da linguagem C, como:

- *stdio.h* e *stdlib.h* para a gestão de ficheiros e memória dinâmica;
- *unistd.h* para verificação de ficheiros e *paths*;
- *string.h* e *ctype.h* para a manipulação de strings e verificação de caracteres;
- *time.h* para o processamento e comparação de datas;
- *stdint.h* para representação de dados (e:. Estado).

Durante o desenvolvimento deste projeto, foram aplicadas boas práticas de **gestão de memória** e **verificação de erros**, recorrendo à ferramenta **Valgrind** para garantir que não ocorressem fugas de memória ou acessos inválidos.

3. Discussão

Durante o desenvolvimento deste projeto surgiram vários desafios relacionados sobretudo com a estrutura do código, a integração de diferentes componentes e a integração do código com a plataforma de testes disponibilizada pelos docentes.

Uma das principais dificuldades sentidas foi a necessidade da utilização da biblioteca **GLib** e, em particular, da estrutura **GHashTable**, que exigiu uma adaptação inicial, nomeadamente no que diz respeito à gestão de memória e à definição de funções de libertação adequadas para cada tipo de dado armazenado. Para além disso, uma parte significativa do trabalho consistiu em garantir que o programa tratava corretamente os casos de erro, valores em falta e situações inesperadas nos ficheiros CSV, o que determinou a criação de funções auxiliares de validação e um controlo rigoroso das mensagens de erro.

A escolha de **GHashTable** para armazenar aeroportos, aeronaves, voos, passageiros e reservatórios foi motivada por:

- tempo de acesso médio *O(1)*, crucial para queries repetidas em milhões de registos;
- comparação robusta de strings;
- gestão automática da memória;
- funções personalizadas para evitar fugas de memória.

Cada input foi mapeado para uma *struct*:

- Aeroporto;
- Aeronave;
- Voo;
- Passageiro;
- Reserva.

Contrariamente a soluções baseadas apenas em arrays de strings, estas estruturas:

- facilitam a validação de cada campo;
- reduzem conversões de tipos;
- permitem extensibilidade futura;
- melhoram a legibilidade do código.

Outro dos desafios mais relevantes foi a adaptação do projeto ao ambiente de testes automático da Unidade Curricular. A plataforma revelou rapidamente discrepâncias entre a execução local e a execução remota, especialmente no que toca à estrutura do repositório, ao comportamento da Makefile e ao formato exato dos outputs exigidos. Estas diferenças obrigaram a várias revisões na organização das pastas, na forma como o executável principal era produzido e até no tratamento de detalhes aparentemente pequenos, como a formatação dos resultados. Este processo, apesar de exigente, permitiu detetar aspetos que poderiam passar despercebidos numa execução exclusivamente local.

Relativamente à utilização do **GitHub**, a colaboração em grupo trouxe alguns desafios ligados à sincronização de ficheiros, conflitos e estrutura do repositório. Contudo, permitiu também o acompanhamento da evolução do trabalho e garantir o registo de todas as alterações.

A **modularização** revelou-se essencial para manter o código organizado. A separação das funcionalidades em diferentes ficheiros *.c* e *.h* tornou mais fácil o isolamento de erros, a reutilização de funções e o garantir de que cada módulo tivesse responsabilidades bem definidas. Ainda assim, surgiram alguns problemas derivados da compatibilidade entre módulos, como referido anteriormente, relativamente à definição e consistência das estruturas usadas por várias componentes do programa.

- ***q1.c*/*q1.h*:** leitura dos dados dos aeroportos, validação dos campos e query 1;
- ***q2.c*/*q2.h*:** gestão dos dados das aeronaves e query 2;
- ***q3.c*/*q3.h***: gestão dos voos e query 3;
- ***read.c*/*read.h***: lê linhas dos ficheiros CSV de forma genérica e faz parsing inicial dos campos;
- ***validators.c*/*validators.h***: contém validações gerais;
- ***csv.c*/*csv.h*:** dá suporte a funções auxiliares de parsing;
- ***erros.c*/*erros.h***: registo estruturado dos erros encontrados nos datasets;
- ***programa-principal.c***: execução das queries e gestão dos ficheiros de entrada/saída;
- ***programa-testes.c***: avaliação do desempenho;

- **<u>Makefile</u>**: compila cada ficheiro do projeto de forma independente.

Cada módulo expõe apenas as funções essenciais através do *.h*, escondendo detalhes internos da implementação. Este **encapsulamento** facilita:

- a substituição futura de algoritmos ou estruturas sem alterar outros ficheiros;
- a manutenção isolada de erros por componente;
- os testes unitários mais simples.

Funções auxiliares foram declaradas como *static* para garantir visibilidade apenas interna ao módulo.

Quanto à metodologia de desenvolvimento do programa, acordamos que uma de nós implementaria as três queries e o programa principal, outro elemento dedicava-se à definição das estruturas de dados para *airports, flights, aircrafts*, *passengers* e *reservations* e à implementação da validação dos datasets e o terceiro membro criava a Makefile, a estrutura/organização das pastas, desenvolvia o programa de testes e garantia a ausência de memory leaks e zero warnings. No final, o objetivo seria verificarmos o trabalho de cada colega e garantir a qualidade e coerência do nosso trabalho.

A análise do desempenho foi feita com o executável *programa-testes*, conforme solicitado, repetindo cada teste 3 vezes para eliminar variações aleatórias nos resultados das medições.

Elemento	**Máquina**	**CPU**	**RAM**	**Sist. Operativo**
a111661	Portátil DELL	Intel Core i5-8365U	16 GB	Ubuntu 24.04
a110903	Portátil HUAWEI	AMD Ryzen 5 3500U	8 GB	Ubuntu 24.04
a111179	Portátil LENOVO	AMD Ryzen 5 753OU	16 GB	Ubuntu 24.04

Tab. 1 - Informações dos Elementos do Grupo

3.1. Metodologia de Medição

Para avaliar o desempenho individual de cada query, desenvolvemos um script bash (*medir_tempos_completo.sh*), disponível no diretório scripts/ do repositório) que automatiza o processo de medição. O script executa cada query isoladamente três vezes, calculando a média aritmética dos tempos obtidos para eliminar variações causadas pelo escalonamento do sistema operativo. Entre execuções, o diretório de resultados é limpo para garantir condições idênticas. A medição temporal utiliza o comando *date +%s%N*, oferecendo precisão de nanossegundos, adequada para queries rápidas.

A tabela seguinte resume o tempo médio de execução de cada query, permitindo avaliar o desempenho do programa em diferentes cenários.

Elemento	Média Query 1	Média Query 2	Média Query 3	Tempo total
a111661	7ms	7ms	7ms	0.894 s
a110903	5635ms	6065ms	5706ms	696.302s
a111179	3103ms	3420ms	3222ms	389.882s

Tab.2 - Tempo de Execução Queries

3.2. Análise da Discrepância de Tempos

Os testes nos três computadores do grupo revelaram variações significativas nos tempos de execução. O portátil DELL (a111661) executou cada query em aproximadamente 7ms, enquanto o HUAWEI (a110903) demorou cerca de 5900ms e o LENOVO (a111179) cerca de 3200ms - diferenças de 843× e 457× respetivamente.

Esta discrepância não reflete ineficiência algorítmica, mas diferenças de hardware, particularmente no tipo de armazenamento. A diferença entre SSD e HDD é determinante: um SSD oferece velocidades de leitura de ~3500 MB/s com latência de 0.1ms, enquanto um HDD atinge apenas ~120 MB/s com latência de 5-10ms. Para o nosso dataset de aproximadamente 500MB, isto resulta numa diferença de

carregamento de 143ms (SSD) versus 4167ms (HDD) antes de qualquer processamento algorítmico.

A validação no AutoTester confirma esta análise: num servidor com hardware uniforme, o tempo total para 120 queries foi de 23.8 segundos (média de 198ms), demonstrando desempenho consistente e adequado. A variação observada nos computadores pessoais é, portanto, exclusivamente atribuível às diferenças de infraestrutura, não à implementação dos algoritmos.

-

4. Conclusão

O desenvolvimento deste projeto permitiu-nos consolidar e aplicar os conhecimentos adquiridos ao longo da Unidade Curricular de Laboratórios de Informática III, nomeadamente na programação em linguagem C, na utilização de estruturas de dados dinâmicas e na gestão eficiente de memória.

Através de uma abordagem modular, foi possível criar um programa organizado, legível e facilmente extensível, capaz de processar grandes volumes de informação provenientes de ficheiros CSV. A utilização da biblioteca **GLib** revelou-se essencial para simplificar a manipulação de estruturas como as HashTables, garantindo um acesso rápido e eficaz aos dados.

Embora a implementação cumpra os requisitos da fase atual, identificaram-se possíveis melhorias, nomeadamente a simplificação de algumas funções de leitura e validação, a criação de funções mais genéricas para evitar repetição de código e a possibilidade de reforçar o tratamento de erros em casos limite.

Estas oportunidades de otimização foram identificadas precisamente no contexto das boas práticas já adotadas, como a verificação sistemática de erros e a validação da integridade da memória, com a ferramenta **Valgrind**. O uso do **Git** e do **GitHub** facilitou a colaboração entre os elementos do grupo, promovendo o controlo de versões e a integração contínua das contribuições de cada membro.

Em suma, o projeto cumpriu os objetivos propostos, proporcionando uma experiência prática completa que envolveu não apenas a implementação técnica, mas também a organização do trabalho em equipa e a utilização de ferramentas de desenvolvimento profissional.